

Framework Semantic Registry Specification

Stage 11D: Generic Natural-Tile Signatures, Explicit Convention Profiles, and LTA Interface Families

mdstats

2026-07-23

Contents

1 Purpose and stage boundary	1
2 Scientific basis and original construction	2
3 Source contract	2
4 Generic tile identity	2
5 Generic ring-interface identity	3
6 Explicit framework profiles	3
7 LTA convention profile	3
8 Public API	4
9 Catalog invariants	4
10 Profile validation	4
11 Resource policy	5
12 Serialization and replay	5
13 Required validation	5
14 Explicit exclusions	5
15 References	6

1 Purpose and stage boundary

Stage 11D adds a persistent semantic layer above certified natural-tiling geometry:

certified tile and window identities

- > derive generic tile face signatures
- > derive generic ring-interface signatures
- > optionally apply one explicit framework convention profile
- > validate profile multiplicities after local classification
- > immutable tile and ring semantic registry

Runtime/API target:

mdstats 0.19.89a0

Primary module:

mdstats/analysis/framework_semantics.py

Stage 11D names structural regions and interfaces. It does not identify ionic minima, merge ring-side anchors into physical states, assign trajectory ions, construct free-energy landscapes, or fit kinetic rates.

2 Scientific basis and original construction

Natural-tile terminology and zeolite natural-tiling semantics follow Blatov, Delgado-Friedrichs, O’Keeffe, and Proserpio [1] and Anurova, Blatov, Ilyushin, and Proserpio [2].

The following are mdstats constructions:

- the canonical machine representation of a tile face signature;
- the generic ring-interface identity consisting of ring order plus the unordered adjacent tile-signature pair;
- explicit rather than automatic application of conventional profiles;
- preservation of oriented side labels alongside the unordered family key;
- post-classification count validation that cannot force or repair labels; and
- source-bound canonical serialization and replay.

3 Source contract

The sole structural source is one immutable `TilingGeometryCatalog`. It already contains:

- dense natural-tile identities;
- dense topological-window identities;
- tile-side incidences;
- ring order;
- periodic relative tile translations; and
- exact source digests.

Stage 11D recomputes every tile face signature from the actual `ring_size` values of its `side_indices`. The pre-existing free-form tile `label` is retained only as `source_label`; it is not trusted as the semantic classifier.

Reference and dynamic oxygen-ring geometry are not required to decide tile or interface semantics. Stages 11E and later join semantics to ring geometry by the persistent `window_index` and `face_digest`.

4 Generic tile identity

For one tile, let n_k be the number of faces bounded by a k -ring. Its canonical face signature is the sorted finite tuple

$$\mathcal{S}_t = ((k_1, n_{k_1}), \dots, (k_m, n_{k_m})), \quad k_1 < \dots < k_m.$$

The canonical symbol is

$$k_1^{n_{k_1}} . k_2^{n_{k_2}} \dots k_m^{n_{k_m}} .$$

Examples are:

4^6

$4^6.6^8$

$4^{12}.6^8.8^6$

The generic machine label is:

tile:<signature>

A generic catalog assigns no conventional framework name.

5 Generic ring-interface identity

For a topological window of order k between tile signatures $\mathcal{S}_a$ and $\mathcal{S}_b$, define

$$\mathcal{S}_R = (k, \text{sort}\{\mathcal{S}_a, \mathcal{S}_b\}).$$

The corresponding machine symbol is:

<k>r:<signature-1>--<signature-2>

This key is unordered because it identifies a structural family. The result also retains, without sorting:

- the original `side_a` and `side_b` records;
- the semantic label of the tile on each oriented side;
- the periodic relative tile translation; and
- the self-image adjacency flag.

The two topological sides therefore remain distinct even when both belong to the same semantic tile family.

6 Explicit framework profiles

Conventional naming is applied only through an explicit `FrameworkSemanticProfile`. A profile contains:

- one exact `TileSemanticRule` for every supported face signature;
- one exact `RingInterfaceRule` for every supported combination of ring order and adjacent semantic tile labels;
- optional expected tile and interface multiplicities; and
- source references.

There is no automatic framework recognition in Stage 11D. Calling the builder without a profile produces only generic machine-readable semantics. Calling it with a profile requires every tile and every interface to match a local rule. Unknown local signatures fail closed.

This boundary prevents a familiar framework label from being inferred solely from a global count coincidence.

7 LTA convention profile

The built-in `LTA_FRAMEWORK_PROFILE` contains the following tile rules.

Face signature	Machine label	Display label	Role	Expected count
$[4^6]$	<code>d4r</code>	D4R	structural unit	6
$[4^6.6^8]$	<code>beta</code>	beta cage	cage	2
$[4^{12}.6^8.8^6]$	<code>alpha</code>	alpha cage	cage	2

The interface rules are:

Ring order	Adjacent labels	Family label	Role	Expected count
4	<code>alpha, d4r</code>	<code>d4r_alpha_4r</code>	exposed 4R	24
4	<code>beta, d4r</code>	<code>d4r_beta_4r</code>	internal 4R	12
6	<code>alpha, beta</code>	<code>alpha_beta_6r</code>	cage interface	16
8	<code>alpha, alpha</code>	<code>alpha_alpha_8r</code>	window	6

The profile does not classify from these counts. First, each tile is classified from its own face signature. Second, each window is classified from its own ring order and adjacent tile labels. Only then are the observed multiplicities compared with the profile expectations.

A count mismatch invalidates the requested profile; it never changes an individual label.

8 Public API

```
build_framework_semantic_catalog(  
    geometry: TilingGeometryCatalog,  
    *,  
    profile: FrameworkSemanticProfile | None = None,  
    resources: FrameworkSemanticsResources | None = None,  
) -> FrameworkSemanticCatalog
```

Built-in profile:

LTA_FRAMEWORK_PROFILE

Primary immutable records:

TileFaceSignature
RingInterfaceSignature
TileSemanticRule
RingInterfaceRule
FrameworkSemanticProfile
SemanticTile
SemanticRingInterface
FrameworkProfileValidation
FrameworkSemanticCatalog

9 Catalog invariants

A valid `FrameworkSemanticCatalog` satisfies:

- tile IDs are dense and ordered;
- window/interface IDs are dense and ordered;
- there is exactly one semantic tile per source tile;
- there is exactly one semantic interface per source window;
- source `face_digest`, side records, periodic translations, and self-adjacency flags are retained exactly;
- generic signatures agree with their source ring orders and tile signatures;
- profile labels are present exactly when a profile is applied;
- profile validation is present exactly when a profile is applied;
- a stored conventional catalog may contain only a matched validation result;
- all records are immutable tuples and frozen dataclasses; and
- the catalog digest covers the source digest, complete profile, all semantic records, and the validation report.

10 Profile validation

`FrameworkProfileValidation` stores:

- all observed tile-label counts;
- all declared expected tile-label counts;
- all observed interface-family counts;
- all declared expected interface-family counts; and
- the final matched flag.

Expected counts may be omitted in a custom profile. Every declared expectation must match; undeclared counts are descriptive rather than constraints. The built-in LTA profile declares all tile and interface counts and is therefore an exact multiplicity gate.

11 Resource policy

`FrameworkSemanticsResources` bounds before classification:

- `max_tiles`;
- `max_windows`; and
- `max_profile_rules`.

A resource failure is transactional. No partial semantic catalog is returned.

12 Serialization and replay

`FrameworkSemanticCatalog.to_dict()` emits canonical JSON-compatible content. `FrameworkSemanticCatalog.from_dict()`

1. reconstructs the complete embedded profile, if present;
2. rebuilds semantics from the supplied `TilingGeometryCatalog`;
3. recomputes local classifications and validation counts; and
4. accepts the payload only when the complete canonical result agrees.

Editing a label, family, profile rule, count, side identity, translation, or source digest therefore fails replay.

13 Required validation

The focused Stage-11D suite must include:

- exact LTA tile signature and semantic-label counts;
- exact LTA 24/12/16/6 interface-family counts;
- preservation of oriented sides and periodic translations;
- generic output without conventional names;
- a deliberately wrong expected count that fails only at validation;
- rejection of the LTA profile on a non-LTA tiling;
- a custom extensible profile;
- resource preflight;
- canonical replay and tamper rejection; and
- public export and constructor validation.

The wider regression boundary must retain Stage 11A–11C, exact and generic natural tiling, primitive rings, periodic cell complexes, net embedding, and framework projection.

14 Explicit exclusions

Stage 11D does not:

- recognize an arbitrary structure as LTA;
- infer semantics from geometry or ion positions;
- use expected counts to choose a framework identity;
- merge topological ring sides into physical ionic states;
- define a species-dependent site center;
- decide whether a cage or portal is accessible;
- assign ions to sites; or
- construct transition rates.

These operations belong to Stage 11E and later stages.

15 References

- [1] V. A. Blatov, O. Delgado-Friedrichs, M. O’Keeffe, and D. M. Proserpio, “Three-periodic nets and tilings: natural tilings for nets”, *Acta Crystallographica Section A* **63**, 418-425 (2007). DOI: [10.1107/S01087667307038287](https://doi.org/10.1107/S01087667307038287).
- [2] N. A. Anurova, V. A. Blatov, G. D. Ilyushin, and D. M. Proserpio, “Natural tilings for zeolite-type frameworks”, *Journal of Physical Chemistry C* **114**, 10160-10170 (2010). DOI: [10.1021/jp1030027](https://doi.org/10.1021/jp1030027).